

B.Sc. (Data Science)

DS101T : Problem Solving and Python Programming

Unit 2: Python Libraries

Mrs. Reshma Masurekar

**Dr. D. Y. Patil Arts, Commerce and Science College,
Pimpri, Pune 18**

2.2 Working with Tabular Numeric Data (Numpy with Python)

- NumPy Arrays Creation Using `array()` Function
- Array Attributes, NumPy Arrays Creation with Initial Placeholder Content
- Integer Indexing, Array Indexing, Boolean ArrayIndexing, Slicing and Iterating in Arrays Basic Arithmetic Operations on NumPy Arrays
- Mathematical Functions in NumPy
- Changing the Shape of an Array, Stacking and Splitting of Arrays, Broadcasting in Arrays.

Practical assignments on Numpy

- a. Assignments on Numpy basics.
- b. Assignments on Numpy Arrays.
- c. Assignments on Numpy Linear Algebra
- d. Assignments on Numpy Statistics
- e. Assignments on Numpy Strings

NumPy

- Numpy : Numerical Python
- Powerful Python library widely used for scientific calculation, Data analysis, and numerical computing task.
- `pip install Numpy`
- In Numpy, Array is the fundamental object.
- Arrays are used to store homogeneous data elements in a contiguous block of memory.

Key Features of NumPy

- **N-Dimensional Arrays:** NumPy's core feature is ndarray, a powerful N-dimensional array object that supports homogeneous data types.
- **Arrays with High Performance:** Arrays are stored in contiguous memory locations, enabling faster computations than Python lists.
- **Broadcasting:** This allows element-wise computations between arrays of different shapes. It simplifies operations on arrays of **various shapes** by automatically aligning their dimensions without creating new data.
- **Vectorization:** Eliminates the need for explicit Python loops by applying operations directly on entire arrays.
- **Linear algebra:** NumPy contains routines for linear algebra operations, such as matrix multiplication, decompositions, and determinants.

Installing NumPy in Python

- To begin using NumPy, you need to install it first. This can be done through pip command:

pip install numpy

- Once installed, import the library with the alias np

import numpy as np

Or

import numpy

NumPy

- NumPy is the fundamental package for numerical computing in Python, offering a powerful array object and a suite of functions for working efficiently with these arrays.
- The advantages of Numpy are:
 - **Multidimensional:** Supports more than just one-dimensional arrays.
 - **Powerful operations:** Includes a vast library of mathematical and statistical functions.
 - **Vectorization:** Allows for operations on entire arrays without the need for explicit loops, enhancing performance.
 - **Integration:** Works seamlessly with other scientific libraries like SciPy, Pandas, and Matplotlib.

Sr. No.	Array	List
1	All elements are of same type	All elements are of different type
2	Elements are stored in a contiguous block of memory.	Elements are not stored in a contiguous block of memory.
3	Arrays are static and cannot be resized once they are created	List can be resized and modified easily.
4	Numpy array takes up less space in memory.	List takes up more space in memory.
5	Support element wise operations.	Do not support element wise operations.

NumPy Arrays Creation Using `array()` Function

- NumPy is used to work with arrays.
- The array object in NumPy is called ndarray.
- NumPy arrays are created using the `array()` function.
- Steps for NumPy Arrays Creation
 - first import NumPy .
 - Then use the `array()` function to create an array. The `array()` function takes a list as an input.

```
import numpy
a1=numpy.array([1,2,3,4,5])
print(a1)
```

Output: [1 2 3 4 5]

```
import numpy as np
a1=np.array([1,2,3,4,5])
print(a1)
```

Output:[1 2 3 4 5]

```
import numpy as np
l1=[1,2,3,4,5]
a1=np.array(l1)
print(a1)
```

Output: [1 2 3 4 5]

```
import numpy as np
l1=[1,2,3,4,5]
a1=np.array(l1)
print(a1)
type(a1)
```

Output:
[1 2 3 4 5]
numpy.ndarray

```
import numpy  
a1=numpy.array([1,2,3,4,5])  
print(a1)
```

Output: [1 2 3 4 5]

This code imports the NumPy library and creates a one-dimensional array called a1 using the numpy.array() function. The array contains the values 1, 2, 3, 4 and 5. Finally, the print() function is used to display the contents of the array.

The import statement is a Python keyword that allows the use of external libraries or modules in a program. In this case, the numpy library is being imported.

The *numpy.array()* function is used to create a new array object. It takes a list or tuple as an argument and returns a new NumPy array. In this case, the list [1, 2, 3, 4,5] is passed as an argument to create the a1 object.

The print() function is used to display the contents of the a1 object. When the code is executed, the output will be [1 2 3 4 5], which represents the contents of the array.

```
import numpy as np  
a1=np.array([1,2,3,4,5])  
print(a1)
```

Output: [1 2 3 4 5]

This code is a simple Python list containing the integers 1 through 5. The list is created using square brackets and separating the elements with commas. The list is ordered, meaning that the first element is 1, the second element is 2, and so on. Lists are a fundamental data structure in Python and are used to store collections of items.

```
print(type(a1))
```

This code uses the Python programming language to print the data type of the variable `a1`. The `type()` function in Python returns the data type of the object passed as an argument. In this case, the `type()` function is applied to the variable `a1`, and the resulting data type is printed to the console using the `print()` function.

Output: numpy.ndarray

- **Creating Arrays**

```
a = np.array([1,2,3])
```

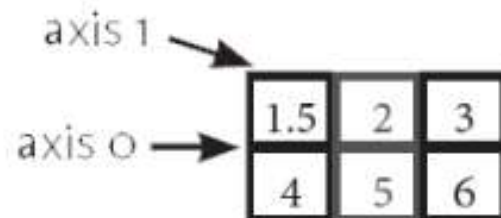
```
b = np.array([(1.5,2,3), (4,5,6)], dtype = float)
```

```
c = np.array([[(1.5,2,3), (4,5,6)],[(3,2,1), (4,5,6)]],  
             dtype = float)
```

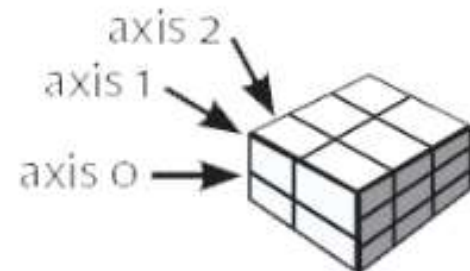
1D array



2D array



3D array



NumPy Array Attributes

- In NumPy, attributes are properties of array objects that provide important information about the arrays and their data.
- In NumPy, attributes are properties of NumPy arrays that provide information about the array's shape, size, data type, dimension, and so on.
- To access the Numpy attributes, `.` Notation is used.

Sr. No.	Attributes	Description
1	ndim	returns number of dimension of the array
2	size	returns number of elements in the array
3	dtype	returns data type of elements in the array
4	shape	returns the size of the array in each dimension.
5	itemsize	returns the size (in bytes) of each elements in the array
6	data	returns the buffer containing actual elements of the array in memory

NumPy Dimensions Attribute

- The **ndim** attribute returns the **number of dimensions (axes) of the array**.
- In NumPy, the dimension of an array is known as its **rank**. Each axis in a NumPy array corresponds to a dimension. The number of axes (dimensions) is referred to as the array's rank.
- Arrays can be of any dimension, from one-dimensional (1D) arrays (also known as vectors) to multi-dimensional arrays like 2D arrays (matrices) or even higher-dimensional arrays.

```
import numpy as np  
a=np.array([1,2,3])  
print(a)  
print(a.ndim)
```

Output: [1 2 3]
1

```
import numpy as np  
a=np.array([[1,2,3],[4,5,6]])  
print(a)  
print(a.ndim)
```

Output: [[1 2 3]
[4 5 6]]
2

NumPy size Attribute

- The NumPy **size** attribute returns the total number of elements in the given array.

```
import numpy as np
a=np.array([[1,2,3],[4,5,6]])
print(a)
print("ndim:",a.ndim)
print("size:",a.size)
```

Output: `[[1 2 3] [4 5 6]]`
ndim: 2
size: 6

NumPy Shape Attribute

- The NumPy **shape** attribute returns a tuple of integers that gives **the size of the array in each dimension**.

```
import numpy as np
a=np.array([[1,2,3],[4,5,6]])
print(a)
print("shape:",a.shape)
```

Output: `[[1 2 3] [4 5 6]]`
`shape: (2, 3)`

resizing an array using the shape attribute in NumPy

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
a.shape = (3,2)
print (a)
```

Output:

NumPy also provides a reshape() function to resize an array

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
b = a.reshape(3,2)
print (b)
```

Output:

NumPy dtype Attribute

- The NumPy dtype attribute to return the datatype of a NumPy array.

```
import numpy as np
a=np.array([[1,2,3],[4,5,6]])
print(a)
print("dtype:",a.dtype)
```

Output:
[[1 2 3] [4 5 6]]
dtype: int64

NumPy itemsize

- The NumPy itemsize attribute returns the size (in bytes) of each element of a NumPy array.

```
import numpy as np
a=np.array([[1,2,3],[4,5,6]])
print(a)
print(a.dtype)
print("Itemsize:",a.itemsize)
```

Output:

```
[[1 2 3] [4 5 6]]
int64
Itemsize: 8
```

- Each element of array is of size 8 bytes.

Numpy attribute example of 1d Numpy array

```
import numpy as np
a=np.array([1,2,3,4,5,6])
print(a)
print("Dimension: ", a.ndim)
print("Size: ", a.size)
print("Shape: ", a.shape)
print("Datatype : ", a.dtype)
print("Itemsize:",a.itemsize)
print("Total bytes:",a.nbytes)
```

Output:

```
[1 2 3 4 5 6]
Dimension: 1
Size: 6
Shape: (6,)
Datatype : int64
Itemsize: 8
Total bytes: 48
```

Numpy attribute example of 2d Numpy array

```
import numpy as np
a=np.array([[1,2,3],[4,5,6]])
print(a)
print("Dimension: ", a.ndim)
print("Size: ", a.size)
print("Shape: ", a.shape)
print("Datatype : ", a.dtype)
print("Itemsize:",a.itemsize)
print("Total bytes:",a.nbytes)
```

Output:

```
[[1 2 3] [4 5 6]]
Dimension: 2
Size: 6
Shape: (2, 3)
Datatype : int64
Itemsize: 8
Total bytes: 48
```

NumPy Arrays Creation with Initial Placeholder Content

- NumPy provides convenient methods to create arrays initialized with specific values like zeros and ones:

Create an empty array

```
import numpy as np
a = np.empty((3, 2))
print("Empty Array:" , a)
print()
```

Output:

```
Empty Array: [[0. 0.]
              [0. 0.]
              [0. 0.]
```

Create an Zeros array

```
import numpy as np
a = np.zeros((3, 2))
print("Zeros Array:" , a)
print()
```

Output:

```
Zeros Array: [[0. 0.]
              [0. 0.]
              [0. 0.]
```

Create an Ones array

```
import numpy as np
a = np.ones((3, 2))
print("Ones Array:" , a)
print()
```

Output:

```
Ones Array: [[1. 1.]
             [1. 1.]
             [1. 1.]
```

Create an arrange array

```
import numpy as np
a = np.arange(10)
print("Arange Array:",a)
```

Output:

```
Arange Array:
[0 1 2 3 4 5 6 7 8 9]
```

Create an Full array

```
full_array = np.full((5), 7)
print("Full Array:")
print(full_array)
print()
```

Output:

```
Full Array: [7 7 7 7 7]
```

Create an Full array

```
import numpy as np
a = np.full((2, 2), 7)
print("Full Array:", a)
print()
```

Output:

```
Full Array: [[7 7]
             [7 7]]
```

Indexing and Slicing

- Indexing a NumPy array means **accessing the elements** of the NumPy array at the given index.
- There are two types of indexing in NumPy: **basic indexing** and **advanced indexing**.
- Slicing a NumPy array means **accessing the subset of the array**. It means **extracting a range of elements** from the data.

Indexing an NumPy Array

- **Array Indexing in NumPy** is used to access or modify specific elements of an array. It allows to retrieve data from arrays by specifying the positions (indices) of elements. It enables efficient data manipulation and analysis for numerical computing tasks.
- **Example:** Index in NumPy array

Element	23	21	55	65	23
Index	0	1	2	3	4
Reverse index	-5	-4	-3	-2	-1

Accessing 1D Array: In a 1D array, we can access elements using their zero-based index:

Accessing element of 1-d array

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
print("Accessing first element of an array:",a[0])
```

Output:

Accessing first element of an
array: 10

Access 2-D Array: To access elements, you specify the row index first and the column index second.

Accessing element of 2d array

```
import numpy as np
a = np.array([[10, 20],[30, 40]])
print("Accessing first element of an  
array:",a[1][1])
```

Output:

Accessing first element of an
array: 40

- **Accessing 3D Arrays:**
- A 3D array can be visualized as a stack of 2D arrays.
- There are three indices:
 - **Depth:** Specifies the 2D slice.
 - **Row:** Specifies the row within the slice.
 - **Column:** Specifies the column within the row.

Accessing element of 1-d array

```
import numpy as np
cube = np.array([[[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]],
                [[10, 11, 12],
                 [13, 14, 15],
                 [16, 17, 18]]])
# Access the element at depth 1, row 2, column0
print(cube[1, 2, 0])
```

Output:

16

Basic Slicing and indexing

- Basic slicing and indexing is used to access a specific element or range of elements from a NumPy array.
- Basic slicing and indexing only return the view of the array.
- Slicing extract a subset of elements using the **start:stop:step** .

Arithmetic operations – 1d array using operators

```
import numpy as np
```

```
arr = np.array([0, 1, 2, 3, 4, 5])
```

```
print(arr[1:4])
```

Output:

```
[1 2 3]
```

Arithmetic operations – 1d array using operators

```
import numpy as np
```

```
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
print(matrix[0:2, 1:3])
```

Output:

```
[[2 3]
```

```
[5 6]]
```

Boolean Indexing

- This indexing has some boolean expressions as the index.
- Boolean indexing allows to filter elements based on conditions.
- Those elements are returned which satisfy that Boolean expression.

Example 1: Using boolean indexing on NumPy array to find numbers greater than 50.

```
import numpy as np  
  
a = np.array([10, 15, 20, 25, 30])  
  
print(arr[arr > 20])
```

Output:

[25 30]

Example 1: Using boolean indexing with multiple conditions to filter elements

```
import numpy as np  
  
arr = np.array([10, 15, 20, 25, 30])  
  
print(arr[(arr > 10) & (arr < 30)])
```

Output:

[15 20 25]

Modifying Array Elements

- NumPy allows direct modification of array elements using indexing:

Example 1: Modify elements in the array using slicing.

```
import numpy as np  
  
a = np.array([1, 2, 3, 4])  
  
a[1:3] = 99  
  
print(a)
```

Output:

```
[ 1 99 99 4]
```

NumPy Arithmetic Array Operations

- NumPy provides a wide range of operations that can perform on arrays, including arithmetic operations.
- NumPy's arithmetic operations are widely used due to their ability to perform simple and efficient calculations on arrays.
- In this tutorial, we will explore some commonly used arithmetic operations in NumPy and learn how to use them to manipulate arrays.

NumPy Arithmetic Array Operations

Element-wise Operation	Operator	Function
Addition	+	add()
Subtraction	-	subtract()
Multiplication	*	multiply()
Division	/	divide()
Exponentiation	**	power()
Modulus	%	mod()

Arithmetic operations – 1d array using operators

```
import numpy as np

a1=np.array([[2,2,2,2],[2,2,2,2]])

a2=np.array([[1,1,1,1],[1,1,1,1]])

print("Addition :",a1+a2)

print("Subtraction :",a1-a2)

print("Multiplication :",a1*a2)

print("Division :",a1/a2)

print("Floor Division :", a1 //a2)

print("Modulo :", a1 % a2)
```

Output:

Addition :
[[3 3 3 3]
[3 3 3 3]]

Subtraction :
[[1 1 1 1]
[1 1 1 1]]

Multiplication :
[[2 2 2 2]
[2 2 2 2]]

Division :
[[2. 2. 2. 2.]
[2. 2. 2. 2.]]

Floor Division :
[[2 2 2 2]
[2 2 2 2]]

Modulo :
[[0 0 0 0]
[0 0 0 0]]

Arithmetic operations – 2d array using operators

```
import numpy as np

a1=np.array([[2,2],[2,2]])
a2=np.array([[1,1],[1,1]])

print("Addition : ",a1+a2)

print("Subtraction : ",a1-a2)

print("Multiplication : ",a1*a2)

print("Division : ",a1/a2)

print("Floor Division : ", a1 //a2)

print("Modulo : ", a1 % a2)

print("Matrix Multiplication : ", a1 @ a2)
```

Output:

```
Addition : [[3 3] [3 3]]
Subtraction : [[1 1] [1 1]]
Multiplication : [[2 2] [2 2]]
Division : [[2. 2.] [2. 2.]]
Floor Division : [[2 2] [2 2]]
Modulo : [[0 0] [0 0]]
Matrix Multiplication : [[4 4] [4 4]]
```

Arithmetic operations – 2d array using function

```
import numpy as np

a1=np.array([[2,2,2,2],[2,2,2,2]])
a2=np.array([[1,1,1,1],[1,1,1,1]])

print("Addition : ",np.add(a1,a2))

print("Subtraction : ",np.subtract(a1,a2))

print("Multiplication : ",np.multiply(a1,a2))

print("Division : ",np.divide(a1,a2))

print("Exponentiation : ", np.power(a1,a2))

print("Modulo : ", np.mod(a1,a2))
```

Output:

```
Addition : [[3 3 3 3] [3 3 3 3]]
Subtraction : [[1 1 1 1] [1 1 1 1]]
Multiplication : [[2 2 2 2] [2 2 2 2]]
Division : [[2. 2. 2. 2.] [2. 2. 2. 2.]]
Exponentiation : [[2 2 2 2] [2 2 2 2]]
Modulo : [[0 0 0 0] [0 0 0 0]]
```

NumPy Statistical Functions

- Statistics involves gathering data, analyzing it, and drawing conclusions based on the information collected. NumPy offers a wide range of statistical functions that allow you to perform various statistical calculations on arrays. These functions can calculate metrics such as mean, median, variance, standard deviation, minimum, maximum, and more.

NumPy Statistical Functions

- **max()**: return the maximum element of an array.
- **min()**: return the minimum element of an array.
- **sum()**: return the sum of all elements of an array.
- **prod()**: return the product of all elements of an array.
- **mean()**: return the mean of an array.
- **median()**: return the median of an array.
- **var()**: return the variance of an array.
- **std()**: return the standard deviation of an array.

max() & min()

- **numpy.max()**: returns maximum value in an array.
- **numpy.min()**: returns minimum value in an array.
- **Example:**

```
import numpy as np
a=np.array([4,2,3,9,7])
Maxval=np.max(a)
Minval=np.min(a)
print("Minimum value:", Minval)
print("Maximum value:", Maxval)
```

sum() and prod()

- **numpy.sum()** : function calculates the sum of all elements in the array.
- **numpy.prod()** : function calculates the product of all elements in the array.

- **Example:**

```
import numpy as np
a=np.array([4,2,3,9,7])
print("Sum : ",np.sum(a))
print("Product : ",np.prod(a))
```

mean()

- **numpy.mean()** : function returns the arithmetic mean of elements in the array. If no axis is specified, it computes the mean of the flattened array. Arithmetic mean is the sum of elements along an axis divided by the number of elements.
- `np.mean(array1)` - calculates the mean over the entire array.
- `np.mean(array1, axis=0)` - calculates the mean along vertical(y-axis) axis.
- `np.mean(array1, axis=1)` calculates the mean along horizontal (x-axis) axis.

Statistical operations – mean 2-d array

```
import numpy as np
a = np.array([[1,2,3],[3,4,5],[4,5,6]])
print("Array is:")
print(a)
print('\n')

print("Applying mean() function:")
print(np.mean(a))
print('\n')

print("Applying mean() function along axis 0:")
print(np.mean(a, axis=0))
print('\n')

print("Applying mean() function along axis 1:")
print(np.mean(a, axis=1))
```

Output:

Array is:

```
[[1 2 3]
 [3 4 5]
 [4 5 6]]
```

Applying mean() function:

```
3.6666666666666665
```

Applying mean() function along axis 0:

```
[2.66666667 3.66666667 4.66666667]
```

Applying mean() function along axis 1:

```
[2. 4. 5.]
```

Statistical operations – mean 2-d array

```
import numpy as np
a = np.array([[1,2,3],[3,4,5],[4,5,6]])
print("Array is:")
print(a)
print('\n')

print("Applying mean() function:")
print(np.mean(a))
print('\n')

print("Applying mean() function along axis 0:")
print(np.mean(a, axis=0))
print('\n')

print("Applying mean() function along axis 1:")
print(np.mean(a, axis=1))
```

Output:

Array is:

```
[[1 2 3]
 [3 4 5]
 [4 5 6]]
```

Applying mean() function:

```
3.6666666666666665
```

Applying mean() function along axis 0:

```
[2.66666667 3.66666667 4.66666667]
```

Applying mean() function along axis 1:

```
[2. 4. 5.]
```

median()

- **numpy.median():** function computes the median along the specified axis. If no axis is specified, it computes the median of the flattened array. Median is defined as the value separating the higher half of a data sample from the lower half
- Median is calculated for 1D array and 2D array.
- In a 2D array, median can be calculated either along the horizontal or the vertical axis individually, or across the entire array.
- When computing the median of a 2D array, we use the axis parameter inside np.median() to specify the axis along which to If we specify,
 - axis = 0, median is calculated along vertical axis.
 - axis = 1, median is calculated along horizontal axis.
- If the axis parameter is not used, the median is computed over the entire array.

Statistical operations on 1-d array

```
import numpy as np
a = np.array([1,2,3,4,5])
print("Our array is:")
print(a)
print("\n")
print("Applying median() function:")
print(np.median(a))
print("\n")
```

Output:

Our array is: [1 2 3 4 5]

Applying median() function: 3.0

Statistical operations on 1-d array

```
import numpy as np
a = np.array([1,2,3,4,5,6])
print("Our array is:")
print(a)
print("\n")
print("Applying median() function:")
print(np.median(a))
print("\n")
```

Output:

Our array is: [1 2 **3 4** 5 6]

Applying median() function: 3.5

Median=(3+4)/2=3.5

Statistical operations on 2-d array

```
import numpy as np
a = np.array([[1,2,3],[3,4,5],[4,5,6]])
print('Our array is:')
print(a)
print("\n")

print("Applying median() function:")
print(np.median(a))
print('\n')

print("Applying median() function along axis 0:")
print(np.median(a, axis=0))
print('\n')

print("Applying median() function along axis 1:")
print(np.median(a, axis=1))
```

Output:

Our array is: [[1 2 3] [3 4 5] [4 5 6]]

Applying median() function: 4.0

Applying median() function along axis 0:
[3. 4. 5.]

Applying median() function along axis 1:
[2. 4. 5.]

Numpy.var() and numpy.std()

- **numpy.var()** function returns the variance of elements in the array. The **variance** is the average of squared deviations, i.e.,
$$\text{var} = \text{mean}(\text{abs}(x - x.\text{mean}())**2).$$

```
import numpy as np
print(np.var([1,2,3,4]))
```

- **numpy.std()** function returns the standard deviation of elements in the array. The **standard deviation** is the square root of the average of squared deviations from the mean, i.e.,
$$\text{std} = \text{sqrt}(\text{mean}(\text{abs}(x - x.\text{mean}())**2)).$$

```
import numpy as np
print(np.std([1,2,3,4]))
```

Statistical operations on 1-d array

```
import numpy as np
a=np.array([[10,20,30,40,50]])
print("Maximum : ",np.max(a))
print("Minimum : ",np.min(a))
print("Sum : ",np.sum(a))
print("Product : ",np.prod(a))
print("Mean : ",np.mean(a))
print("Median : ",np.median(a))
print("Variance : ",np.var(a))
print("Standard Deviation : ",np.std(a))
```

Output:

```
Maximum : 50
Minimum : 10
Sum : 150
Product : 12000000
Mean : 30.0
Median : 30.0
Variance : 200.0
Standard Deviation : 14.142135623730951
```

Statistical operations on 2-d array

```
import numpy as np
a=np.array([[1,2],[3,4]])
print("Maximum : ",np.max(a))
print("Minimum : ",np.min(a))
print("Sum : ",np.sum(a))
print("Product : ",np.prod(a))
print("Mean : ",np.mean(a))
print("Median : ",np.median(a))
print("Variance : ",np.var(a))
print("Standard Deviation : ",np.std(a))
```

Output:

```
Maximum : 4
Minimum : 1
Sum : 10
Product : 24
Mean : 2.5
Median : 2.5
Variance : 1.25
Standard Deviation : 1.118033988749895
```

Broadcasting

- Broadcasting simplifies mathematical operations on arrays with different shapes.
- It enables NumPy to efficiently apply operations element-wise without explicitly copying or reshaping data.

Example 1 : Broadcasting Array in Single Value and 1D Addition

```
import numpy as np  
  
arr = np.array([1, 2, 3])  
  
res = arr + 1  
  
# Adds 1 to each element  
  
print(res)
```

Output: [2 3 4]

Example 1 : Broadcasting Array in 1D and 2D Addition

```
import numpy as np  
  
a1 = np.array([2, 4, 6])  
  
a2 = np.array([[1, 3, 5], [7, 9, 11]]) res = a1 +  
a2  
  
print(res)
```

Output: [[3 7 11]
[9 13 17]]

Example 1 : Broadcasting Array in Single Value and 1D Addition

```
import numpy as np  
  
matrix = np.array([[1, 2], [3, 4]])  
  
vector = np.array([10, 20])  
  
result = matrix * vector  
  
print(result)
```

Output: [[10 40] [30 80]]

Difference between one-dimensional, two-dimensional, and multi-dimensional arrays.

Parameter	One-Dimensional Array	Two-Dimensional Array	Multi-Dimensional Array
Definition	A flat array with elements in a single row.	A table-like structure with rows and columns.	Arrays with more than two axes.
Shape Example	<code>(n,)</code> (e.g., <code>(3,)</code>)	<code>(m, n)</code> (e.g., <code>(2, 3)</code>)	<code>(x, y, z)</code> (e.g., <code>(2, 3, 4)</code>)
Use Case	Simple lists or sequences.	Matrices, images.	3D data like videos or tensors.
Example Code	<code>np.array([1, 2, 3])</code>	<code>np.array([[1, 2], [3, 4]])</code>	<code>np.random.rand(2, 3, 4)</code>
Access Example	<code>array[1]</code>	<code>array[1, 0]</code>	<code>array[1, 0, 2]</code>

Differences between slicing and indexing in NumPy?

Parameter	Indexing	Slicing
Definition	Accessing a specific element in an array using its position.	Extracting a portion or subset of elements from an array.
Result Type	Returns a single value or a smaller array depending on the input.	Always returns a new view or copy of the array.
Syntax	Uses square brackets with integers or boolean values.	Uses colons (:) to define ranges.
Usage	Suitable for accessing individual elements.	Used for extracting larger portions of an array.
Example	<code>array[1]</code> for a 1D array, or <code>array[1, 2]</code> for a 2D array.	<code>array[1:4]</code> extracts elements from index 1 to 3.